Handling Missing Data

General Principles

In many real-world datasets, some observations or parameters may be missing or partially known. In a standard frequentist approach, these missing values are often dropped—a practice known as *complete case analysis*. While sometimes justifiable, ignoring missing data frequently leads to biased estimates and a loss of valuable information McElreath (2018).

In the Bayesian framework, **missing data are simply treated as unknown parameters to be estimated within the model itself**. The generative model fundamentally does not distinguish between unobserved parameters (e.g., regression coefficients) and unobserved data. The entire joint distribution is evaluated, allowing the model to smoothly "impute" the missing values based on the observed data and prior structural relationships Stan Development Team (2024).

Conceptually, the probability model (likelihood) designed for the measurement constraint serves as the prior distribution for the unobserved data.

Missingness Mechanisms

From a causal inference perspective, the strategy for handling missing data hinges on the underlying mechanism causing the missingness McElreath (2018). Imagine evaluating an animal's true lifespan or waiting time until adoption (H) based on some underlying trait like coat color or species (S). However, a missingness process (D)—such as a sensor failing, a researcher ending the study, or the trailing event occurring unobserved—might result in an incomplete, partially observed record (H^*):

1. **Missing Completely At Random (MCAR)**: *Random sensor failure.* The missingness (D) is entirely independent of the animal's traits (S) or its true lifespan (H).
    - **Strategy**: Dropping incomplete cases (complete case analysis) is okay and generally unbiased, but it results in a loss of statistical efficiency.

2. **Missing At Random (MAR)**: *Missingness conditional on traits.* The missingness (D) is systematically related to other observed variables, such as the animal's species or behavior (S) affecting the likelihood of consistent tracking.

    - **Strategy**: Statistical inference can be rescued by correctly conditioning on the structural causes of the missingness (e.g., S) to close backdoor estimation paths.

3. **Not Missing At Random (NMAR)**: *Missingness caused by the outcome itself.* The probability of missingness (D) is directly caused by the unobserved value itself (H) (e.g., animals that wait the longest to be adopted outlast the study duration and are therefore missing/right-censored).

    - **Strategy**: Usually hopeless unless we can explicitly model the missingness mechanism (the censoring process), such as right-censored waiting times in survival analysis.

Considerations

> **Note**

- **Parameters vs. Data**: In generative models, there is no philosophical distinction between "unobserved parameters" and "unobserved data". Both are simply unknowns governed by probability distributions. The likelihood of an observed data point behaves identically to the prior for an unobserved one.
- **Imputation vs. Marginalization**: BayesForge offers two primary computational strategies. **Imputation** treats each missing value as a parameter, yielding full posterior distributions for the unknowns; it is highly flexible and transparent but computationally intensive. **Marginalization** mathematically integrates over the unknowns, which is often faster but can be analytically opaque or numerically unstable for certain density functions.
- **JAX Arrays are Immutable**: `BayesForge` uses JAX in the backend. JAX arrays do not support direct heterogeneous assignments (e.g., `y[ii_mis] = y_mis`). When splicing missing data into a larger array, you must use `.at[...].set(...)` operations.
- **Partial Knowledge**: Sometimes covariance or correlation matrices are only partially known. Instead of discarding the known parts, we only sample the unknown cells uniformly within mathematically valid positive-definite bounds and reconstruct the matrix deterministically.

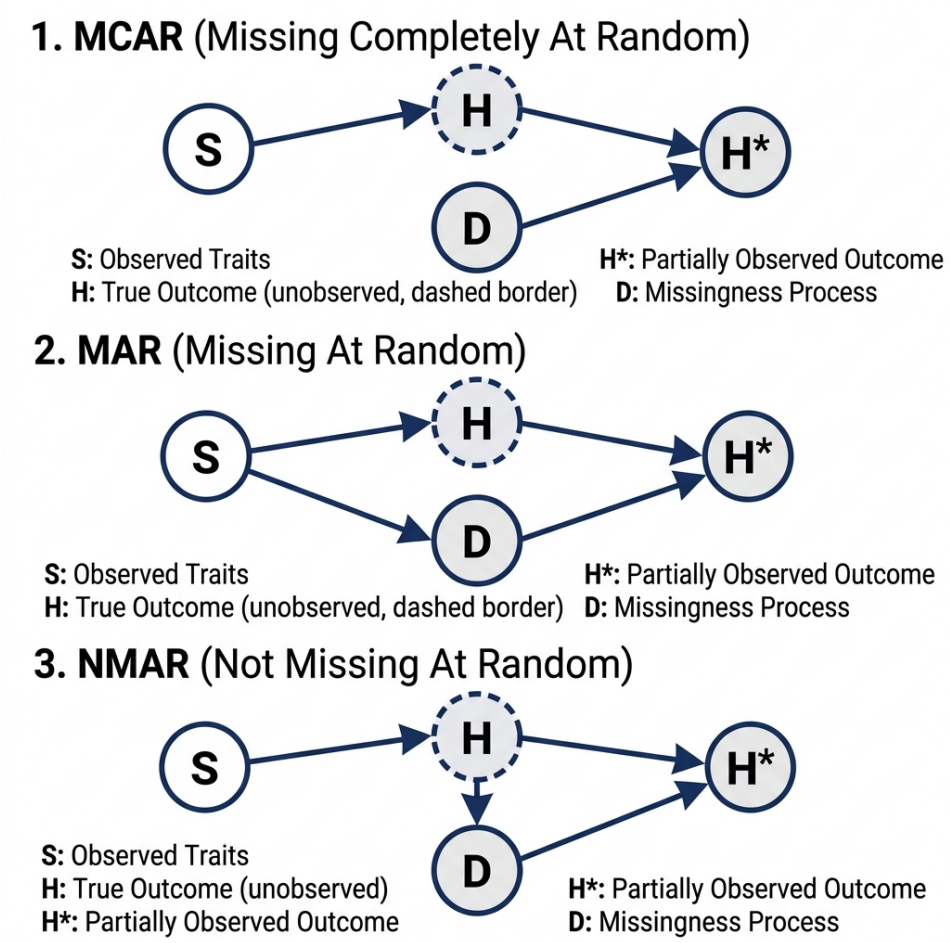


Figure 1: Missingness Mechanisms: MCAR, MAR, and NMAR DAGs. S represents Observed Traits, H represents the True Outcome (unobserved), H* represents the Partially Observed Outcome, and D represents the Missingness Process.

Examples

Below are four typical missing data patterns translated into the **BayesForge (BF)** framework. Each example demonstrates a specific strategy discussed in the General Principles and Considerations sections.

1. Simple Missing Imputation

This example demonstrates the core concept of **Imputation** vs. **Marginalization** (from *Considerations*). We generate a linear regression dataset (**DGP**), **mask** 5 outcome values, **infer** both the model parameters and the 5 missing outcomes simultaneously as latent variables, and **compare** the posterior estimates against the true values.

Python

```python
from BayesForge import bf
import jax.numpy as jnp
import numpy as np

m = bf(platform="cpu")

# 1. DGP: Simulate linear regression data with known parameters
np.random.seed(42)
N = 60
true_alpha, true_beta, true_sigma = 2.0, 1.5, 0.4
x = jnp.array(np.random.normal(0.0, 1.0, size=N))
y_true = true_alpha + true_beta * x + jnp.array(np.random.normal(0.0, true_sigma, size=N))

# 2. Masking: Mask the first 5 observations
N_mis = 5
x_mis, y_mis_true = x[:N_mis], y_true[:N_mis]
x_obs, y_obs = x[N_mis:], y_true[N_mis:]

m.data_on_model = dict(x_obs=x_obs, y_obs=y_obs, x_mis=x_mis, N_mis=N_mis)

# 3. Inference: Jointly estimate parameters and impute missing outcomes
def missing_data_model(x_obs, y_obs, x_mis, N_mis):
    alpha = m.dist.normal(0, 10, name="alpha")
    beta = m.dist.normal(0, 10, name="beta")
    sigma = m.dist.half_normal(5, name="sigma")
```

```
    # Impute missing outcomes from their covariates x_mis
    with m.dist.plate("missing_obs", N_mis):
        y_mis = m.dist.normal(alpha + beta * x_mis, sigma, name="y_mis")

    # Likelihood for observed data
    m.dist.normal(alpha + beta * x_obs, sigma, obs=y_obs)

m.fit(missing_data_model, progress_bar=False)
summ = m.summary()

# 4. Comparison: Estimated vs True values
print(f"alpha: True = {true_alpha:.2f}, Estimated = {float(summ.loc['alpha', 'mean']):.2f}")
print(f"beta:  True = {true_beta:.2f}, Estimated = {float(summ.loc['beta', 'mean']):.2f}")
print(f"sigma: True = {true_sigma:.2f}, Estimated = {float(summ.loc['sigma', 'mean']):.2f}")
for i in range(N_mis):
    est_yi = float(summ.loc[f"y_mis[{i}]", "mean"])
    print(f"y_mis[{i}]: True = {float(y_mis_true[i]):.2f}, Imputed = {est_yi:.2f}")
```

R

```
library(BayesForge)
m <- importBF(platform = "cpu")
jnp <- reticulate::import("jax.numpy")

# 1. DGP: Simulate linear regression data with known parameters
set.seed(42)
N <- 60L
true_alpha <- 2.0
true_beta <- 1.5
true_sigma <- 0.4
x_r <- rnorm(N, 0.0, 1.0)
y_true_r <- true_alpha + true_beta * x_r + rnorm(N, 0.0, true_sigma)

# 2. Masking: Mask the first 5 observations
N_mis <- 5L
x_mis <- jnp$array(x_r[1:N_mis])
y_mis_true <- y_true_r[1:N_mis]
x_obs <- jnp$array(x_r[(N_mis + 1):N])
y_obs <- jnp$array(y_true_r[(N_mis + 1):N])

m$data_on_model <- list(x_obs = x_obs, y_obs = y_obs, x_mis = x_mis, N_mis = N_mis)
```

```r
# 3. Inference: Jointly estimate parameters and impute missing outcomes
missing_data_model <- function(x_obs, y_obs, x_mis, N_mis) {
    alpha <- m$dist$normal(0, 10, name = "alpha")
    beta <- m$dist$normal(0, 10, name = "beta")
    sigma <- m$dist$half_normal(5, name = "sigma")

    y_mis <- m$dist$normal(alpha + beta * x_mis, sigma, shape = reticulate::tuple(N_mis), nam
    m$dist$normal(alpha + beta * x_obs, sigma, obs = y_obs)
}

m$fit(missing_data_model, progress_bar = FALSE)
summ <- m$summary()

# 4. Comparison: Estimated vs True values
print(summ)
print(y_mis_true)
```

Julia

```julia
using PyBayesForge
using PythonCall
using Random

m = importBF(platform="cpu")
jnp = pyimport("jax.numpy")

# 1. DGP: Simulate linear regression data with known parameters
Random.seed!(42)
N = 60
true_alpha, true_beta, true_sigma = 2.0, 1.5, 0.4
x_jl = randn(N)
y_true_jl = true_alpha .+ true_beta .* x_jl .+ true_sigma .* randn(N)

# 2. Masking: Mask the first 5 observations
N_mis = 5
x_mis = jnp.array(x_jl[1:N_mis])
y_mis_true = y_true_jl[1:N_mis]
x_obs = jnp.array(x_jl[(N_mis + 1):N])
y_obs = jnp.array(y_true_jl[(N_mis + 1):N])
```

```
m.data_on_model = pydict(Dict("x_obs" => x_obs, "y_obs" => y_obs, "x_mis" => x_mis, "N_mis" =

# 3. Inference: Jointly estimate parameters and impute missing outcomes
@BF function missing_data_model(x_obs, y_obs, x_mis, N_mis)
    alpha = m.dist.normal(0, 10, name="alpha")
    beta = m.dist.normal(0, 10, name="beta")
    sigma = m.dist.half_normal(5, name="sigma")

    m.dist.normal(alpha + beta * x_mis, sigma, shape=(N_mis,), name="y_mis")
    m.dist.normal(alpha + beta * x_obs, sigma, obs=y_obs)
end

m.fit(missing_data_model, progress_bar=false)
summ = m.summary()

# 4. Comparison: Estimated vs True values
println(summ)
println("True masked y: ", y_mis_true)
```

2. Partially Known Parameters (Covariance)

This example illustrates how to handle **Partial Knowledge** (from *Considerations*). We generate 2D Gaussian observations (**DGP**), **mask** the off-diagonal covariance while keeping the diagonal variances known, **infer** the unknown covariance within positive-definite bounds, and **compare** posterior estimates against the true mean and covariance.

Python

```
from BayesForge import bf
import jax.numpy as jnp

m = bf(platform="cpu")

# 1. DGP: Simulate 2D observations with known variances (2.0, 1.5) and true cov = 0.8
true_mu = jnp.array([1.0, -1.0])
var1, var2, true_cov = jnp.float64(2.0), jnp.float64(1.5), 0.8
true_Sigma = jnp.array([[2.0, true_cov], [true_cov, 1.5]])
y = m.dist.multivariate_normal(loc=true_mu, covariance_matrix=true_Sigma, shape=(100,), sampl

# 2. Masking: Provide only var1 and var2 as known; cov is unobserved
```

```
m.data_on_model = dict(y=y, var1=var1, var2=var2)

# 3. Inference: Sample unknown cov within positive-definite bounds
def partially_known_model(y, var1, var2):
    max_cov = jnp.sqrt(var1 * var2)
    min_cov = -max_cov

    mu = m.dist.normal(0, 5, shape=(2,), name="mu")
    cov = m.dist.uniform(min_cov, max_cov, name="cov")

    row1 = jnp.stack([var1, cov])
    row2 = jnp.stack([cov, var2])
    Sigma = jnp.stack([row1, row2])

    m.dist.multivariate_normal(loc=mu, covariance_matrix=Sigma, obs=y)

m.fit(partially_known_model, progress_bar=False)
summ = m.summary()

# 4. Comparison: Estimated vs True values
print(f"mu[0]: True = {float(true_mu[0]):.2f}, Estimated = {float(summ.loc['mu[0]', 'mean'])
print(f"mu[1]: True = {float(true_mu[1]):.2f}, Estimated = {float(summ.loc['mu[1]', 'mean'])
print(f"cov:   True = {true_cov:.2f}, Estimated = {float(summ.loc['cov', 'mean']):.2f}")
```

R

```
library(BayesForge)
m <- importBF(platform = "cpu")
jnp <- reticulate::import("jax.numpy")

# 1. DGP: Simulate 2D observations with known variances (2.0, 1.5) and true cov = 0.8
true_mu <- jnp$array(c(1.0, -1.0))
true_cov <- 0.8
true_Sigma <- jnp$array(list(c(2.0, true_cov), c(true_cov, 1.5)))
y <- m$dist$multivariate_normal(loc = true_mu, covariance_matrix = true_Sigma, shape = c(100
var1 <- jnp$float64(2.0)
var2 <- jnp$float64(1.5)

# 2. Masking: Provide only var1 and var2 as known; cov is unobserved
m$data_on_model <- list(y = y, var1 = var1, var2 = var2)
```

```r
# 3. Inference: Sample unknown cov within positive-definite bounds
partially_known_model <- function(y, var1, var2) {
    max_cov <- jnp$sqrt(var1 * var2)
    min_cov <- -max_cov

    mu <- m$dist$normal(0, 5, shape = c(2L), name = "mu")
    cov <- m$dist$uniform(min_cov, max_cov, name = "cov")

    row1 <- jnp$stack(list(var1, cov))
    row2 <- jnp$stack(list(cov, var2))
    Sigma <- jnp$stack(list(row1, row2))

    m$dist$multivariate_normal(loc = mu, covariance_matrix = Sigma, obs = y)
}

m$fit(partially_known_model, progress_bar = FALSE)
summ <- m$summary()

# 4. Comparison: Estimated vs True values
print(summ)
print(c(true_mu_0 = 1.0, true_mu_1 = -1.0, true_cov = true_cov))
```

Julia

```julia
using PyBayesForge
using PythonCall

m = importBF(platform="cpu")
jnp = pyimport("jax.numpy")

# 1. DGP: Simulate 2D observations with known variances (2.0, 1.5) and true cov = 0.8
true_cov = 0.8
true_Sigma = jnp.array([[2.0, true_cov], [true_cov, 1.5]])
y = m.dist.multivariate_normal(loc=jnp.array([1.0, -1.0]), covariance_matrix=true_Sigma, sha
var1 = jnp.float64(2.0)
var2 = jnp.float64(1.5)

# 2. Masking: Provide only var1 and var2 as known; cov is unobserved
m.data_on_model = pydict(Dict("y" => y, "var1" => var1, "var2" => var2))

# 3. Inference: Sample unknown cov within positive-definite bounds
```

```
@BF function partially_known_model(y, var1, var2)
    max_cov = jnp.sqrt(var1 * var2)
    min_cov = -max_cov

    mu = m.dist.normal(0, 5, shape=(2,), name="mu")
    cov = m.dist.uniform(min_cov, max_cov, name="cov")

    row1 = jnp.stack([var1, cov])
    row2 = jnp.stack([cov, var2])
    Sigma = jnp.stack([row1, row2])

    m.dist.multivariate_normal(loc=mu, covariance_matrix=Sigma, obs=y)
end

m.fit(partially_known_model, progress_bar=false)
summ = m.summary()

# 4. Comparison: Estimated vs True values
println(summ)
println("True mu = [1.0, -1.0], True cov = ", true_cov)
```

3. Sliced Missing Data (Time Series)

This example highlights the technical implementation detail of **JAX Arrays are Immutable** (from *Considerations*). We generate a random-walk time series (**DGP**), **mask** 3 interior time steps, use `.at[...].set(...)` to splice the imputed latent variables into the full series during **inference**, and **compare** the imputed states against the true masked trajectory values.

Python

```
from BayesForge import bf
import jax.numpy as jnp

m = bf(platform="cpu")

# 1. DGP: Simulate a random-walk time series of length 25 with known step noise
N_total = 25
true_sigma = 0.5
steps = m.dist.normal(0.0, true_sigma, shape=(N_total,), sample=True, seed=42)
y_true = jnp.cumsum(steps)
```

```python
# 2. Masking: Mask 3 interior time steps (indices 4, 10, 15)
ii_mis = jnp.array([4, 10, 15], dtype=jnp.int32)
ii_obs = jnp.array([i for i in range(N_total) if i not in (4, 10, 15)], dtype=jnp.int32)
y_obs_ts = y_true[ii_obs]
y_mis_true = y_true[ii_mis]
N_mis = int(ii_mis.shape[0])

m.data_on_model = dict(y_obs=y_obs_ts, ii_obs=ii_obs, N_mis=N_mis, ii_mis=ii_mis, N_total=N_t

# 3. Inference: Splice observed and latent missing points via JAX .at[].set()
def sliced_missing_model(y_obs, ii_obs, N_mis, ii_mis, N_total):
    sigma = m.dist.gamma(1, 1, name="sigma")
    y_mis = m.dist.normal(0, 10, shape=(N_mis,), name="y_mis")

    y_full = jnp.zeros(N_total)
    y_full = y_full.at[ii_obs].set(y_obs)
    y_full = y_full.at[ii_mis].set(y_mis)

    m.dist.normal(0, 10, obs=y_full[0], name="y0")
    m.dist.normal(y_full[:-1], sigma, obs=y_full[1:], name="y_step")

m.fit(sliced_missing_model, progress_bar=False)
summ = m.summary()

# 4. Comparison: Estimated vs True masked time steps
print(f"sigma: True = {true_sigma:.2f}, Estimated = {float(summ.loc['sigma', 'mean']):.2f}")
for k, idx in enumerate([4, 10, 15]):
    est_val = float(summ.loc[f"y_mis[{k}]", "mean"])
    print(f"t={idx} (y_mis[{k}]): True = {float(y_mis_true[k]):.2f}, Imputed = {est_val:.2f}"
```

R

```r
library(BayesForge)
m <- importBF(platform = "cpu")
jnp <- reticulate::import("jax.numpy")

# 1. DGP: Simulate a random-walk time series of length 25 with known step noise
N_total <- 25L
true_sigma <- 0.5
steps <- m$dist$normal(0.0, true_sigma, shape = c(N_total), sample = TRUE, seed = 42L)
```

```r
y_true <- jnp$cumsum(steps)

# 2. Masking: Mask 3 interior time steps (indices 4, 10, 15)
ii_mis <- jnp$array(c(4L, 10L, 15L), dtype = jnp$int32)
ii_obs <- jnp$array(setdiff(0L:(N_total - 1L), c(4L, 10L, 15L)), dtype = jnp$int32)
y_obs_ts <- y_true[ii_obs]
y_mis_true <- y_true[ii_mis]
N_mis <- 3L

m$data_on_model <- list(y_obs = y_obs_ts, ii_obs = ii_obs, N_mis = N_mis, ii_mis = ii_mis, N_

# 3. Inference: Splice observed and latent missing points via JAX .at[].set()
sliced_missing_model <- function(y_obs, ii_obs, N_mis, ii_mis, N_total) {
    sigma <- m$dist$gamma(1, 1, name = "sigma")
    y_mis <- m$dist$normal(0, 10, shape = c(N_mis), name = "y_mis")

    y_full <- jnp$zeros(N_total)
    y_full <- y_full$at[ii_obs]$set(y_obs)
    y_full <- y_full$at[ii_mis]$set(y_mis)

    m$dist$normal(0, 10, obs = y_full[0L], name = "y0")
    m$dist$normal(y_full[0L:(N_total - 2L)], sigma, obs = y_full[1L:(N_total - 1L)], name = "
}

m$fit(sliced_missing_model, progress_bar = FALSE)
summ <- m$summary()

# 4. Comparison: Estimated vs True masked time steps
print(summ)
print(y_mis_true)
```

Julia

```julia
using PyBayesForge
using PythonCall

m = importBF(platform="cpu")
jnp = pyimport("jax.numpy")

# 1. DGP: Simulate a random-walk time series of length 25 with known step noise
N_total = 25
```

```
true_sigma = 0.5
steps = m.dist.normal(0.0, true_sigma, shape=(N_total,), sample=true, seed=42)
y_true = jnp.cumsum(steps)

# 2. Masking: Mask 3 interior time steps (indices 4, 10, 15)
ii_mis = jnp.array([4, 10, 15], dtype=jnp.int32)
ii_obs = jnp.array([i for i in 0:(N_total-1) if !(i in (4, 10, 15))], dtype=jnp.int32)
y_obs_ts = y_true[ii_obs]
y_mis_true = y_true[ii_mis]
N_mis = 3

m.data_on_model = pydict(Dict("y_obs" => y_obs_ts, "ii_obs" => ii_obs, "N_mis" => N_mis, "ii

# 3. Inference: Splice observed and latent missing points via JAX .at[].set()
@BF function sliced_missing_model(y_obs, ii_obs, N_mis, ii_mis, N_total)
    sigma = m.dist.gamma(1, 1, name="sigma")
    y_mis = m.dist.normal(0, 10, shape=(N_mis,), name="y_mis")

    y_full = jnp.zeros(N_total)
    y_full = y_full.at[ii_obs].set(y_obs)
    y_full = y_full.at[ii_mis].set(y_mis)

    m.dist.normal(0, 10, obs=y_full[0], name="y0")
    m.dist.normal(y_full[pyslice(0, -1)], sigma, obs=y_full[pyslice(1, nothing)], name="y_ste
end

m.fit(sliced_missing_model, progress_bar=false)
summ = m.summary()

# 4. Comparison: Estimated vs True masked time steps
println(summ)
println("True masked y_mis: ", y_mis_true)
```

4. Censored Data via Survival Analysis (NMAR)

This example corresponds to the **Not Missing At Random (NMAR)** mechanism (from *Missingness Mechanisms* in *General Principles*). We generate waiting times from an exponential proportional-hazards **DGP**, **mask (right-censor)** any waiting time that exceeds the study cutoff $C = 3.0$ (so missingness is directly caused by the outcome itself), **infer** the log-hazard parameters by marginalizing over the censored tail, and **compare** the estimated parameters against the ground truth.

Python

```python
from BayesForge import bf
import jax.numpy as jnp
import numpy as np

m = bf(platform="cpu")

# 1. DGP: Simulate true exponential waiting times with covariate x
np.random.seed(42)
N = 200
true_alpha, true_beta = -0.5, 0.8
x = np.random.normal(0.0, 1.0, size=N)
true_rate = np.exp(true_alpha + true_beta * x)
t_true = np.random.exponential(scale=1.0 / true_rate)

# 2. Masking (NMAR Right-Censoring): Study ends at cutoff C = 3.0
cutoff = 3.0
event = (t_true <= cutoff).astype(np.int32)   # 1 = observed event, 0 = right-censored (missi
t_obs = np.minimum(t_true, cutoff)            # observed exposure time

m.data_on_model = dict(
    x=jnp.array(x),
    t_obs=jnp.array(t_obs),
    event=jnp.array(event),
)

# 3. Inference: Marginalize right-censored observations via survival likelihood
def censored_survival_model(x, t_obs, event):
    alpha = m.dist.normal(0, 5, name="alpha")
    beta = m.dist.normal(0, 5, name="beta")
    hazard = jnp.exp(alpha + beta * x)
    # Exact log-likelihood for exponential survival with right-censoring at t_obs
    m.dist.poisson(rate=t_obs * hazard, obs=event)

m.fit(censored_survival_model, progress_bar=False)
summ = m.summary()

# 4. Comparison: Estimated vs True values
print(f"Censored fraction: {1.0 - event.mean():.1%}")
print(f"alpha: True = {true_alpha:.2f}, Estimated = {float(summ.loc['alpha', 'mean']):.2f}")
print(f"beta:  True = {true_beta:.2f}, Estimated = {float(summ.loc['beta', 'mean']):.2f}")
```

R

```r
library(BayesForge)
m <- importBF(platform = "cpu")
jnp <- reticulate::import("jax.numpy")

# 1. DGP: Simulate true exponential waiting times with covariate x
set.seed(42)
N <- 200L
true_alpha <- -0.5
true_beta <- 0.8
x_r <- rnorm(N, 0.0, 1.0)
true_rate <- exp(true_alpha + true_beta * x_r)
t_true <- rexp(N, rate = true_rate)

# 2. Masking (NMAR Right-Censoring): Study ends at cutoff C = 3.0
cutoff <- 3.0
event_r <- as.integer(t_true <= cutoff)
t_obs_r <- pmin(t_true, cutoff)

m$data_on_model <- list(
    x = jnp$array(x_r),
    t_obs = jnp$array(t_obs_r),
    event = jnp$array(event_r, dtype = jnp$int32)
)

# 3. Inference: Marginalize right-censored observations via survival likelihood
censored_survival_model <- function(x, t_obs, event) {
    alpha <- m$dist$normal(0, 5, name = "alpha")
    beta <- m$dist$normal(0, 5, name = "beta")
    hazard <- jnp$exp(alpha + beta * x)
    m$dist$poisson(rate = t_obs * hazard, obs = event)
}

m$fit(censored_survival_model, progress_bar = FALSE)
summ <- m$summary()

# 4. Comparison: Estimated vs True values
print(summ)
print(c(true_alpha = true_alpha, true_beta = true_beta))
```

Julia

```julia
using PyBayesForge
using PythonCall
using Random

m = importBF(platform="cpu")
jnp = pyimport("jax.numpy")

# 1. DGP: Simulate true exponential waiting times with covariate x
Random.seed!(42)
N = 200
true_alpha, true_beta = -0.5, 0.8
x_jl = randn(N)
true_rate = exp.(true_alpha .+ true_beta .* x_jl)
t_true = randexp(N) ./ true_rate

# 2. Masking (NMAR Right-Censoring): Study ends at cutoff C = 3.0
cutoff = 3.0
event_jl = Int32.(t_true .<= cutoff)
t_obs_jl = min.(t_true, cutoff)

m.data_on_model = pydict(Dict(
    "x" => jnp.array(x_jl),
    "t_obs" => jnp.array(t_obs_jl),
    "event" => jnp.array(event_jl, dtype=jnp.int32)
))

# 3. Inference: Marginalize right-censored observations via survival likelihood
@BF function censored_survival_model(x, t_obs, event)
    alpha = m.dist.normal(0, 5, name="alpha")
    beta = m.dist.normal(0, 5, name="beta")
    hazard = jnp.exp(alpha + beta * x)
    m.dist.poisson(rate=t_obs * hazard, obs=event)
end

m.fit(censored_survival_model, progress_bar=false)
summ = m.summary()

# 4. Comparison: Estimated vs True values
println(summ)
println("True alpha = ", true_alpha, ", True beta = ", true_beta)
```

Mathematical Details

Bayesian Formulation

Missing data imputation assumes that the underlying generative process connects both the observed variables Y_{obs} and the missing variables Y_{mis}.

For a simple Gaussian imputation, the joint posterior of the parameters (μ, σ) and the missing data Y_{mis} is proportional to:

$$P(\mu, \sigma, Y_{\text{mis}}|Y_{\text{obs}}) \propto P(Y_{\text{obs}}|\mu, \sigma)\, P(Y_{\text{mis}}|\mu, \sigma)\, P(\mu, \sigma)$$

Because both Y_{obs} and Y_{mis} are generated from the identical normal distribution Normal(μ, σ), the sampler naturally infers values for Y_{mis} that are highly plausible given the parameters μ, σ fitted to Y_{obs}. Conceptually, the likelihood function $P(Y_{\text{mis}}|\mu, \sigma)$ acts as the structural prior for the missing observations Y_{mis}. By sampling Y_{mis} during execution, we perform **Bayesian Imputation**, averaging over the uncertainty of the missing values.

Marginalization

Alternatively, **Marginalization** analytically integrates out the missing values, removing the need to estimate thousands of imputed parameters. For unobserved variables, the likelihood of the observed dataset is evaluated by integrating over all possible values of the missing data:

$$P(Y_{\text{obs}}|\mu, \sigma) = \int P(Y_{\text{obs}}, Y_{\text{mis}}|\mu, \sigma)\, dY_{\text{mis}}$$

While marginalization is computationally more expedient by shrinking the parameter space, it can be analytically challenging and numerically unstable for models with extreme density tails. In models where missingness limits are known, such as right-censored survival data, marginalization relies on tools like the Complementary Cumulative Distribution Function (CCDF) to calculate the remaining cumulative probability beyond the censoring threshold.

Constraints

For partially known covariance matrices, the reconstructed parameter Σ is mathematically bound by the positive-definite limits of a 2×2 matrix, leading to:

$$\det(\Sigma) > 0 \implies \text{var}_1\text{var}_2 - \text{cov}^2 > 0 \implies |\text{cov}| < \sqrt{\text{var}_1\text{var}_2}$$

Model validation tests

- Imputation example: `0.Code.py` discrete imputation reproduced from the NumPyro tutorial.
- Documentation examples: the Python, R and Julia examples of this page are run by the documentation tests (`Python/test_model_pages.py`, `R/test_r_model_pages.py`, `Julia/test_julia_model_pages.py`).

Notes

> **Note**
>
> - **Missing Completely At Random (MCAR) vs Not Missing At Random (NMAR)**: The structural examples shown above effectively operate under the assumption that the probability of data being missing is independent of the value itself (MCAR or MAR). If the missingness mechanism depends on the missing value itself (NMAR)—e.g., people with low income refusing to report their income, or right-censored animal adoption waiting durations—a secondary likelihood must explicitly model the probability of the data being missing or censored to prevent inferential bias.
> - **Censored Data**: Censoring is a special form of missingness where partial information is retained (e.g., an observation is known to exceed a specific threshold, $D > D_{\text{censor}}$). By explicitly truncating the imputation prior or utilizing the CCDF, the generative model extracts information from the censored record without distorting rate estimates.

McElreath, Richard. 2018. *Statistical Rethinking: A Bayesian course with examples in R and Stan*. Chapman; Hall/CRC.

Stan Development Team. 2024. *Stan User's Guide: Missing Data*. https://mc-stan.org/docs/stan-users-guide/missing-data.html.